

Министерство науки и высшего образования Российской Федерации

федеральное государственное автономное
образовательное учреждение высшего образования
«Самарский национальный исследовательский университет имени
академика С.П. Королева»

Институт информатики и кибернетики Кафедра
технической кибернетики

Отчет по лабораторной работе №3

Дисциплина: «ООП»

Выполнил: Булавкина Виктория Станиславовна

Группа: 6201-120303D

Проверил: преподаватель Борисов Д.С.

Самара, 2025

Задание 2

```
FunctionPointIndexOutOfBoundsException.java × InappropriateFunctionPointException.java © LinkedListTabulatedFunction.java
1 package functions;
2
3 public class FunctionPointIndexOutOfBoundsException extends IndexOutOfBoundsException { 13 usages viktoriabulavkina7-sys
4     public FunctionPointIndexOutOfBoundsException() { 6 usages viktoriabulavkina7-sys
5         super();
6     }
7
8     public FunctionPointIndexOutOfBoundsException(String message) { 17 usages viktoriabulavkina7-sys
9         super(message);
10    }
11 }
```

```
1 package functions;
2
3 public class InappropriateFunctionPointException extends Exception { 21 usages viktoriabulavkina7-sys
4     public InappropriateFunctionPointException() { 8 usages viktoriabulavkina7-sys
5         super();
6     }
7
8     public InappropriateFunctionPointException(String message) { 16 usages viktoriabulavkina7-sys
9         super(message);
10    }
11 }
```

Созданы два специализированных класса исключений в пакете functions:

- `FunctionPointIndexOutOfBoundsException` — исключение выхода за границы набора точек при обращении к ним по номеру, наследуется от класса `IndexOutOfBoundsException`;
- `InappropriateFunctionPointException` — исключение, возникающее при попытке добавить или изменить точку функции ненадлежащим образом.

Задание 3

```

25 public ArrayTabulatedFunction(double leftX, double rightX, double[] values) { 6 usages  viktoriabulavkina7-sys
26     if (leftX >= rightX) {
27         throw new IllegalArgumentException("Левая граница должна быть меньше правой границы: " + leftX + " >= " + rightX);
28     }
29     if (values.length < 2) {
30         throw new IllegalArgumentException("Количество баллов должно быть не менее 2: " + values.length);
31     }
32
33     this.pointsCount = values.length;
34     this.points = new FunctionPoint[pointsCount + 5];
35     double step = (rightX - leftX) / (pointsCount - 1);
36     for (int i = 0; i < pointsCount; i++) {
37         double x = leftX + i * step;
38         points[i] = new FunctionPoint(x, values[i]);
39     }
40 }
41
42 @ public ArrayTabulatedFunction(FunctionPoint[] points) { 6 usages  viktoriabulavkina7-sys
43     if (points.length < 2) {
44         throw new IllegalArgumentException("Требуется как минимум 2 точки");
45     }
46
47     this.pointsCount = points.length;
48     this.points = new FunctionPoint[pointsCount + 5];
49     for (int i = 0; i < pointsCount; i++) {
50         this.points[i] = new FunctionPoint(points[i]);
51     }
52 }

```

Реализация конструкторов класса ArrayTabulatedFunction с проверками корректности аргументов

```

    if (index < 0 || index >= pointsCount) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс: " + index + ", Количеств: " + pointsCount);
    }
    return points[index].getY();
}

```

Данная проверка добавлена в начале методов getPoint, setPoint, getPointX, setPointX, getPointY, setPointY и deletePoint.

```
241 @Override 6 usages viktoriabulavkina7-sys *
242 public FunctionPoint getPoint(int index) {
243     FunctionNode node = getNodeByIndex(index);
244     return new FunctionPoint(node.getPoint());
245 }
246
247 @Override 2 usages viktoriabulavkina7-sys *
248 public void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException {
249     if (index < 0 || index >= size) {
250         throw new FunctionPointIndexOutOfBoundsException("Индекс выходит за пределы: " + index);
251     }
252
253     // Проверка порядка X с учетом эпсилона
254     if (index > 0 && point.getX() <= getPoint(index - 1).getX() - EPSILON) {
255         throw new InappropriateFunctionPointException("X должен строго возрастать");
256     }
257     if (index < size - 1 && point.getX() >= getPoint(index + 1).getX() + EPSILON) {
258         throw new InappropriateFunctionPointException("X должен строго возрастать");
259     }
260
261     FunctionNode node = getNodeByIndex(index);
262     node.setPoint(new FunctionPoint(point));
263 }
264
265 @Override 3 usages viktoriabulavkina7-sys
266 public double getPointX(int index) {
267     return getPoint(index).getX();
268 }
```

```

@Override 1 usage viktoriabulavkina7-sys
public void setPointX(int index, double x) throws InappropriateFunctionPointException {
    FunctionPoint point = getPoint(index);
    setPoint(index, new FunctionPoint(x, point.getY()));
}

@Override 2 usages viktoriabulavkina7-sys
public double getPointY(int index) {
    return getPoint(index).getY();
}

@Override 1 usage viktoriabulavkina7-sys *
public void setPointY(int index, double y) {
    FunctionNode node = getNodeByIndex(index);
    node.setPoint(new FunctionPoint(node.getPoint().getX(), y));
}

@Override 4 usages viktoriabulavkina7-sys
public void deletePoint(int index) {
    deleteNodeByIndex(index);
}

```

```

@Override 6 usages viktoriabulavkina7-sys *
public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
    // Находим позицию для вставки
    int insertIndex = size;
    for (int i = 0; i < size; i++) {
        double currentX = getPointX(i);
        if (Math.abs(currentX - point.getX()) < EPSILON) {
            throw new InappropriateFunctionPointException("Точка с X=" + point.getX() + " уже существует");
        }
        if (currentX > point.getX() + EPSILON) {
            insertIndex = i;
            break;
        }
    }

    FunctionNode newNode = addNodeByIndex(insertIndex);
    newNode.setPoint(new FunctionPoint(point));
}

```

В методы `setPoint()` и `setPointX()` дополнительно внедрена проверка корректности значения координаты `x`, обеспечивающая её нахождение между соседними точками для сохранения монотонности области определения.

В метод `addPoint()` добавлена проверка на отсутствие точки с таким же значением `x` во избежание дублирования координат в области определения

Задание 4

```
1 package functions;
2
3 public class LinkedListTabulatedFunction implements TabulatedFunctionImp { 7 usages viktoriabulavkina7-sys *
4     // Внутренний класс для узлов списка
5     private static class FunctionNode { 29 usages viktoriabulavkina7-sys
6         private FunctionPoint point; 4 usages
7         private FunctionNode prev; 2 usages
8         private FunctionNode next; 2 usages
9
10        public FunctionNode(FunctionPoint point) { 2 usages viktoriabulavkina7-sys
11            this.point = point;
12        }
13        public FunctionNode(double x, double y) { no usages viktoriabulavkina7-sys
14            this.point = new FunctionPoint(x, y);
15        }
16        public FunctionPoint getPoint() { 10 usages viktoriabulavkina7-sys
17            return point;
18        }
19        public void setPoint(FunctionPoint point) { 5 usages viktoriabulavkina7-sys
20            this.point = point;
21        }
22        public FunctionNode getPrev() { 6 usages viktoriabulavkina7-sys
23            return prev;
24        }
25        public void setPrev(FunctionNode prev) { 8 usages viktoriabulavkina7-sys
26            this.prev = prev;
27        }
28        public FunctionNode getNext() { 5 usages viktoriabulavkina7-sys
29            return next;
30        }
31        public void setNext(FunctionNode next) { 8 usages viktoriabulavkina7-sys
32            this.next = next;
33        }
34    }
35 }
```

Основной класс `LinkedListTabulatedFunction` содержит поля для управления списком: `head` представляет фиктивный узел циклического списка, `pointsCount` хранит количество точек для избежания пересчета, `lastAccessedNode` и `lastAccessedIndex` реализуют механизм кэширования последнего доступа для оптимизации операций.

```

// Поля основного класса
private FunctionNode head; // Голова списка (не содержит данных) 17 usages
private int size; // Количество значащих элементов 22 usages
private FunctionNode lastAccessedNode; // Для оптимизации доступа 3 usages
private int lastAccessedIndex; // Индекс последнего доступного узла 7 usages
private static final double EPSILON = 1e-10; // Машинный эпсилон 10 usages

// Инициализация списка с помощью

```

Конструкторы класса полностью соответствуют ArrayTabulatedFunction и выполняют необходимые проверки параметров, включая проверку что левая граница меньше правой и количество точек не менее двух, после чего инициализируют список через вспомогательные методы.

```

53 // Метод для получения узла по индексу с оптимизацией
54 private FunctionNode getNodeByIndex(int index) { 7 usages  viktoriabulavkina7-sys *
55     if (index < 0 || index >= size) {
56         throw new FunctionPointIndexOutOfBoundsException("Индекс выходит за пределы: " + index);
57     }
58
59     // Оптимизация: начинаем поиск с последнего доступного узла
60     FunctionNode current;
61     int startIndex;
62     if (lastAccessedIndex != -1 && Math.abs(index - lastAccessedIndex) < Math.min(index, size - index)) {
63         // Ближе к последнему доступному узлу
64         current = lastAccessedNode;
65         startIndex = lastAccessedIndex;
66     } else if (index < size / 2) {
67         // Ближе к началу
68         current = head.getNext();
69         startIndex = 0;
70     } else {
71         // Ближе к концу
72         current = head.getPrev();
73         startIndex = size - 1;
74     }
75
76     // Поиск нужного узла
77     if (index > startIndex) {
78         for (int i = startIndex; i < index; i++) {
79             current = current.getNext();
80         }
81     } else if (index < startIndex) {
82         for (int i = startIndex; i > index; i--) {
83             current = current.getPrev();
84         }
85     }

```

```

87         // Сохраняем для будущей оптимизации
88         lastAccessedNode = current;
89         lastAccessedIndex = index;
90         return current;
91     }

```

Метод `getNodeByIndex` реализует оптимизированный доступ к элементам списка с использованием двухуровневой оптимизации: сначала проверяется возможность доступа от последнего `accessed` узла, затем выбирается оптимальное направление обхода - с начала или с конца списка в зависимости от позиции искомого элемента.

```

93 // Метод для добавления узла по индексу
94 @private FunctionNode addNodeByIndex(int index) { 2 usages  viktoriabulavkina7-sys *
95     if (index < 0 || index > size) {
96         throw new FunctionPointIndexOutOfBoundsException("Индекс выходит за пределы: " + index);
97     }
98
99     FunctionNode newNode = new FunctionNode(new FunctionPoint(x: 0, y: 0));
100
101     if (size == 0) {
102         // Первый элемент
103         newNode.setNext(head);
104         newNode.setPrev(head);
105         head.setNext(newNode);
106         head.setPrev(newNode);
107     } else if (index == size) {
108         // Вставка в конец
109         FunctionNode last = head.getPrev();
110         last.setNext(newNode);
111         newNode.setPrev(last);
112         newNode.setNext(head);
113         head.setPrev(newNode);
114     } else {
115         // Вставка в середину
116         FunctionNode current = getNodeByIndex(index);
117         FunctionNode prevNode = current.getPrev();
118         prevNode.setNext(newNode);
119         newNode.setPrev(prevNode);
120         newNode.setNext(current);
121         current.setPrev(newNode);
122     }
123 }

```

Метод `addNodeByIndex` обеспечивает вставку нового узла в указанную позицию списка с корректным обновлением связей между узлами и поддержанием актуальности кэшированной информации о последнем доступе.


```
// Метод для удаления узла по индексу
private FunctionNode deleteNodeByIndex(int index) { 1 usage 2 viktoriabulavkina7-sys *
    if (index < 0 || index >= size) {
        throw new FunctionPointIndexOutOfBoundsException("Индекс выходит за пределы: " + index);
    }

    if (size < 3) {
        throw new IllegalStateException("Невозможно удалить точку — требуется минимум 2 точки");
    }

    FunctionNode nodeToDelete = getNodeByIndex(index);
    FunctionNode prevNode = nodeToDelete.getPrev();
    FunctionNode nextNode = nodeToDelete.getNext();

    prevNode.setNext(nextNode);
    nextNode.setPrev(prevNode);

    size--;
    lastAccessedIndex = -1; // Сбрасываем кэш
    return nodeToDelete;
}
```

Метод `deleteNodeByIndex` выполняет удаление узла по указанному индексу с проверкой что после удаления останется минимально необходимое количество точек, а также корректно обновляет кэш последнего доступа при необходимости.

Задание 5

```
156 // Конструкторы
157 @ public LinkedListTabulatedFunction(double[] xValues, double[] yValues) { 28 usages viktoriabulavkina7-sys
158     if (xValues.length != yValues.length) {
159         throw new IllegalArgumentException("Массивы должны иметь одинаковую длину");
160     }
161     if (xValues.length < 2) {
162         throw new IllegalArgumentException("Требуется как минимум 2 точки");
163     }
164
165     initializeList();
166     for (int i = 0; i < xValues.length; i++) {
167         addNodeToTail().setPoint(new FunctionPoint(xValues[i], yValues[i]));
168     }
169 }
170
171 @ public LinkedListTabulatedFunction(FunctionPoint[] points) { 22 usages viktoriabulavkina7-sys
172     if (points.length < 2) {
173         throw new IllegalArgumentException("Требуется как минимум 2 точки");
174     }
175
176     initializeList();
177     for (FunctionPoint point : points) {
178         addNodeToTail().setPoint(point);
179     }
180 }
```

Конструкторы класса `LinkedListTabulatedFunction` принимают параметры левой и правой границ области определения и количество точек либо массив значений, при этом выполняют строгую проверку входных данных на корректность - убеждаются что левая граница действительно меньше правой и что количество точек соответствует минимальному требованию не менее двух, что является обязательным условием для существования табулированной функции. В случае нарушения этих условий конструкторы выбрасывают `IllegalArgumentException` с соответствующим сообщением об ошибке, после чего инициализируют внутреннее состояние объекта через вспомогательные методы `initialize` и `initializeWithValues`, которые создают начальную структуру связанного списка с равномерно распределенными точками на заданном интервале.

```

199  @Override 3 usages viktoriabulavkina7-sys *
200  public double getFunctionValue(double x) {
201      // Проверяем границы с учетом эпсилона
202      if (x < getLeftDomainBorder() - EPSILON || x > getRightDomainBorder() + EPSILON) {
203          return Double.NaN;
204      }
205
206      // Поиск точки с точно таким же X
207      for (int i = 0; i < size; i++) {
208          FunctionNode node = getNodeByIndex(i);
209          if (Math.abs(node.getPoint().getX() - x) < EPSILON) {
210              return node.getPoint().getY();
211          }
212      }
213
214      // Поиск интервала для интерполяции
215      for (int i = 0; i < size - 1; i++) {
216          FunctionNode current = getNodeByIndex(i);
217          FunctionNode next = current.getNext();
218
219          double x1 = current.getPoint().getX();
220          double x2 = next.getPoint().getX();
221          double y1 = current.getPoint().getY();
222          double y2 = next.getPoint().getY();
223
224          if (x >= x1 - EPSILON && x <= x2 + EPSILON) {
225              if (Math.abs(x2 - x1) < EPSILON) {
226                  return y1; // Вертикальный отрезок
227              }
228              // Линейная интерполяция
229              return y1 + (y2 - y1) * (x - x1) / (x2 - x1);
230          }
231      }

```

Метод `getFunctionValue` демонстрирует преимущества прямой работы со структурой списка путем выполнения прямого последовательного обхода для поиска интервала содержащего x , что эффективнее использования `getNodeByIndex` для данной операции.

```

292 @Override 6 usages viktoriabulavkina7-sys *
293 public void addPoint(FunctionPoint point) throws InappropriateFunctionPointException {
294     // Находим позицию для вставки
295     int insertIndex = size;
296     for (int i = 0; i < size; i++) {
297         double currentX = getPointX(i);
298         if (Math.abs(currentX - point.getX()) < EPSILON) {
299             throw new InappropriateFunctionPointException("Точка с X=" + point.getX() + " уже существует");
300         }
301         if (currentX > point.getX() + EPSILON) {
302             insertIndex = i;
303             break;
304         }
305     }
306
307     FunctionNode newNode = addNodeByIndex(insertIndex);
308     newNode.setPoint(new FunctionPoint(point));
309 }
310 }

```

Метод `addPoint` оптимизирован за счет прямого обхода списка для проверки уникальности координаты X и поиска позиции вставки, что позволяет избежать многократных вызовов `getNodeByIndex` и снижает сложность операции.

```

43 // Инициализация пустого списка с головой
44 private void initializeList() { 2 usages viktoriabulavkina7-sys
45     head = new FunctionNode(new FunctionPoint(x: 0, y: 0));
46     head.setNext(head);
47     head.setPrev(head);
48     size = 0;
49     lastAccessedNode = head;
50     lastAccessedIndex = -1;
51 }

```

Методы `initList` и выполняет инициализацию списка при создании объекта, создавая циклическую структуру с равномерным распределением точек по заданному интервалу и устанавливая начальные значения для кэша доступа.

Задание 6

```
1 package functions;
2
3 public interface TabulatedFunctionImp { 12 usages 2 implementations viktoriabulavkina7-sys
4     // Методы доступа к области определения
5     double getLeftDomainBorder(); 3 usages 2 implementations viktoriabulavkina7-sys
6     double getRightDomainBorder(); 3 usages 2 implementations viktoriabulavkina7-sys
7     double getFunctionValue(double x); 3 usages 2 implementations viktoriabulavkina7-sys
8
9     // Методы работы с точками
10    int getPointsCount(); 8 usages 2 implementations viktoriabulavkina7-sys
11    FunctionPoint getPoint(int index); 6 usages 2 implementations viktoriabulavkina7-sys
12    void setPoint(int index, FunctionPoint point) throws InappropriateFunctionPointException; 2 usages 2 implementations viktoriabulavkina7-sys
13    double getPointX(int index); 3 usages 2 implementations viktoriabulavkina7-sys
14    void setPointX(int index, double x) throws InappropriateFunctionPointException; 1 usage 2 implementations viktoriabulavkina7-sys
15    double getPointY(int index); 2 usages 2 implementations viktoriabulavkina7-sys
16    void setPointY(int index, double y); 1 usage 2 implementations viktoriabulavkina7-sys
17    void deletePoint(int index); 4 usages 2 implementations viktoriabulavkina7-sys
18    void addPoint(FunctionPoint point) throws InappropriateFunctionPointException; 6 usages 2 implementations viktoriabulavkina7-sys
19 }
```

Создан интерфейс `TabulatedFunction`, определяющий контракт для работы с табулированными функциями. Интерфейс объявляет следующие методы:

- `getPoint()`, `setPoint()` - получение и модификация точек
- `getPointX()`, `setPointX()`, `getPointY()`, `setPointY()` - работа с координатами
- `addPoint()`, `deletePoint()` - добавление и удаление точек
- `getLeftDomainBorder()`, `getRightDomainBorder()` - границы области определения
- `getPointsCount()` - количество точек
- `getFunctionValue()` - вычисление значения функции»

Задание 7

Проверяем

Main:

```

1 import functions.*;
2
3 public class Main {
4     public static void main(String[] args) {
5         System.out.println("=== ТЕСТИРОВАНИЕ ARRAY TABULATED FUNCTION ===");
6         testFunction(new ArrayTabulatedFunction( leftX: 0.1, rightX: 10.1, pointsCount: 6));
7         System.out.println("\n\n=== ТЕСТИРОВАНИЕ LINKED LIST TABULATED FUNCTION ===");
8         testFunction(new LinkedListTabulatedFunction(new double[]{0.1, 2.1, 4.1, 6.1, 8.1, 10.1}, new double[]{0, 0, 0, 0, 0, 0}));
9         System.out.println("\n\n=== ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ ===");
10        testExceptions();
11        System.out.println("\n\n=== ДОПОЛНИТЕЛЬНЫЕ ТЕСТЫ ===");
12        testAdditionalCases();
13    }
14
15    @ private static void testFunction(TabulatedFunctionImp function) {
16        System.out.println("Функция log10(x):");
17        System.out.println("Область определения: [" + function.getLeftDomainBorder() + "; " + function.getRightDomainBorder() + "]");
18        System.out.println("Количество точек: " + function.getPointsCount());
19        System.out.println("Тип функции: " + function.getClass().getSimpleName());
20
21        // Устанавливаем значения y = log10(x)
22        System.out.println("\nТочки функции:");
23        for (int i = 0; i < function.getPointsCount(); i++) {
24            double x = function.getPointX(i);
25            double y = Math.log10(x);
26            function.setPointY(i, y);
27            System.out.printf("Точка %d: (%.1f; %.3f)%n", i + 1, x, y);
28        }
29
30        System.out.println("\nЗначения функции в различных точках:");
31        double[] testPoints = {-1, 0, 0.1, 0.3, 0.5, 1, 1.5, 2, 3, 5, 7, 10, 10.1, 11};
32        for (double x : testPoints) {
33            double y = function.getFunctionValue(x);
34            if (Double.isNaN(y)) {
35                System.out.printf("f(%.1f) = не определено%n", x);

```

```

36         } else {
37             double exact = Math.log10(x);
38             System.out.printf("f(%.1f) = %.3f (точное: %.3f)%n", x, y, exact);
39         }
40     }
41
42     // Тестирование операций с точками
43     try {
44         System.out.println("\nДобавляем точку (3; 0.477):");
45         function.addPoint(new FunctionPoint( x: 3, Math.log10(3)));
46         System.out.println("Количество точек после добавления: " + function.getPointsCount());
47
48         System.out.println("\nДобавляем точку (7; 0.845):");
49         function.addPoint(new FunctionPoint( x: 7, Math.log10(7)));
50         System.out.println("Количество точек после добавления: " + function.getPointsCount());
51
52         System.out.println("\nУдаляем точку с индексом 0:");
53         function.deletePoint( index: 0);
54         System.out.println("Количество точек после удаления: " + function.getPointsCount());
55
56         System.out.println("\nИтоговая информация о точках:");
57         for (int i = 0; i < function.getPointsCount(); i++) {
58             double x = function.getPointX(i);
59             double y = function.getPointY(i);
60             System.out.printf("Точка %d: (%.1f; %.3f)%n", i + 1, x, y);
61         }
62     } catch (Exception e) {
63         System.out.println("Ошибка при работе с точками: " + e.getMessage());
64     }
65 }
66
67 private static void testExceptions() { 1 usage  viktoriabulavkina7-sys *
68     System.out.println("1. Тестирование недопустимых индексов:");

```

```

69     try {
70         TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 0, rightX: 10, pointsCount: 5);
71         func.getPoint( index: 10); // Неверный индекс
72     } catch (FunctionPointIndexOutOfBoundsException e) {
73         System.out.println("    Поймано исключение: " + e.getMessage());
74     }
75
76     System.out.println("\n2. Тестирование нарушения порядка X:");
77     try {
78         TabulatedFunctionImp func = new LinkedListTabulatedFunction(new double[]{0.5, 1.5, 2.5}, new double[]{1, 2, 3});
79         func.setPoint( index: 1, new FunctionPoint( x: 0.1, y: 5));
80     } catch (InappropriateFunctionPointException e) {
81         System.out.println("    Поймано исключение: " + e.getMessage());
82     }
83
84     System.out.println("\n3. Тестирование дублирования точек:");
85     try {
86         TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 0, rightX: 4, pointsCount: 3);
87         // Добавляем точку с X=2.0, который уже существует в равномерной сетке [0, 2, 4]
88         func.addPoint(new FunctionPoint( x: 2.0, y: 5.0));
89     } catch (InappropriateFunctionPointException e) {
90         System.out.println("    Поймано исключение: " + e.getMessage());
91     }
92
93     System.out.println("\n4. Тестирование удаления при минимальном количестве точек:");
94     try {
95         TabulatedFunctionImp func = new LinkedListTabulatedFunction(
96             new double[]{0, 1}, new double[]{0, 1});
97         func.deletePoint( index: 0); // Попытка удалить точку когда всего 2 точки
98     } catch (IllegalStateException e) {
99         System.out.println("    Поймано исключение: " + e.getMessage());
100     }

```

```

102     System.out.println("\n5. Тестирование неверных границ области определения:");
103     try {
104         TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 10, rightX: 0, pointsCount: 5); // Левая граница больше правой
105     } catch (IllegalArgumentException e) {
106         System.out.println("    Поймано исключение: " + e.getMessage());
107     }
108
109     System.out.println("\n6. Тестирование недостаточного количества точек:");
110     try {
111         TabulatedFunctionImp func = new LinkedListTabulatedFunction(new double[]{0}, new double[]{0}); // Всего 1 точка
112     } catch (IllegalArgumentException e) {
113         System.out.println("    Поймано исключение: " + e.getMessage());
114     }
115
116     System.out.println("\n7. Тестирование установки недопустимого X:");
117     try {
118         TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 0.5, rightX: 2.5, pointsCount: 3);
119         func.setPointX( index: 1, x: 0.1); // X=0.1 меньше предыдущего X=0.5
120     } catch (InappropriateFunctionPointException e) {
121         System.out.println("    Поймано исключение: " + e.getMessage());
122     }
123
124     System.out.println("\n8. Тестирование получения точки с неверным индексом:");
125     try {
126         TabulatedFunctionImp func = new LinkedListTabulatedFunction(
127             new double[]{0, 1, 2}, new double[]{0, 1, 2});
128         func.getPointY( index: -1); // Отрицательный индекс
129     } catch (FunctionPointIndexOutOfBoundsException e) {
130         System.out.println("    Поймано исключение: " + e.getMessage());
131     }
132
133     System.out.println("\n9. Тестирование особых случаев вычисления функции:");
134     TabulatedFunctionImp func = new ArrayTabulatedFunction( leftX: 1, rightX: 10, pointsCount: 5);

```



```

135     System.out.println("    f(-5) = " + func.getFunctionValue(x: -5)); // Вне области определения
136     System.out.println("    f(15) = " + func.getFunctionValue(x: 15)); // Вне области определения
137 }
138
139 private static void testAdditionalCases() { 1 usage 2 viktoriabulavkina7-sys *
140     System.out.println("\n1. Тестирование добавления и удаления точки (0,0):");
141     try {
142         ArrayTabulatedFunction func = new ArrayTabulatedFunction(leftX: -2, rightX: 3, pointsCount: 6);
143         func.addPoint(new FunctionPoint(x: 0, y: 0));
144         System.out.println("    Точка (0,0) успешно добавлена в ArrayTabulatedFunction");
145         func.deletePoint(index: func.getPointsCount() - 1);
146         System.out.println("    Последняя точка успешно удалена");
147     } catch (Exception e) {
148         System.out.println("    Ошибка: " + e.getMessage());
149     }
150
151     System.out.println("\n2. Тестирование добавления и удаления точки (0,0) в LinkedList:");
152     try {
153         LinkedListTabulatedFunction func = new LinkedListTabulatedFunction(
154             new double[]{-2, -1, 1, 2}, new double[]{4, 1, 1, 4});
155         func.addPoint(new FunctionPoint(x: 0, y: 0));
156         System.out.println("    Точка (0,0) успешно добавлена в LinkedListTabulatedFunction");
157         func.deletePoint(index: func.getPointsCount() - 1);
158         System.out.println("    Последняя точка успешно удалена");
159     } catch (Exception e) {
160         System.out.println("    Ошибка: " + e.getMessage());
161     }
162
163     System.out.println("\n3. Тестирование работы с машинным эпсилон:");
164     try {

```

```

165         ArrayTabulatedFunction func = new ArrayTabulatedFunction(leftX: 0, rightX: 2, pointsCount: 3);
166         // Пытаемся добавить точку с  $x = 1.0 + 1e-10$ , что почти равно существующей
167         func.addPoint(new FunctionPoint(x: 1.0 + 1e-10, y: 5.0));
168         System.out.println("    Точка добавлена (разница больше машинного эпсилон)");
169     } catch (InappropriateFunctionPointException e) {
170         System.out.println("    Исключение: " + e.getMessage());
171     }
172 }
173 }

```

```
C:\Users\Виктория\.jdk\openjdk-25.0.1\bin\java.exe "-javaag
=== ТЕСТИРОВАНИЕ ARRAY TABULATED FUNCTION ===
Функция log10(x):
Область определения: [0.1; 10.1]
Количество точек: 6
Тип функции: ArrayTabulatedFunction

Точки функции:
Точка 1: (0,1; -1,000)
Точка 2: (2,1; 0,322)
Точка 3: (4,1; 0,613)
Точка 4: (6,1; 0,785)
Точка 5: (8,1; 0,908)
Точка 6: (10,1; 1,004)

Значения функции в различных точках:
f(-1,0) = не определено
f(0,0) = не определено
f(0,1) = -1,000 (точное: -1,000)
f(0,3) = -0,868 (точное: -0,523)
f(0,5) = -0,736 (точное: -0,301)
f(1,0) = -0,405 (точное: 0,000)
f(1,5) = -0,074 (точное: 0,176)
f(2,0) = 0,256 (точное: 0,301)
f(3,0) = 0,453 (точное: 0,477)
f(5,0) = 0,690 (точное: 0,699)
f(7,0) = 0,841 (точное: 0,845)
f(10,0) = 1,000 (точное: 1,000)
f(10,1) = 1,004 (точное: 1,004)
f(11,0) = не определено

Добавляем точку (3; 0.477):
Количество точек после добавления: 7
```

Добавляем точку (7; 0.845):

Количество точек после добавления: 8

Удаляем точку с индексом 0:

Количество точек после удаления: 7

Итоговая информация о точках:

Точка 1: (2,1; 0,322)

Точка 2: (3,0; 0,477)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (7,0; 0,845)

Точка 6: (8,1; 0,908)

Точка 7: (10,1; 1,004)

=== ТЕСТИРОВАНИЕ LINKED LIST TABULATED FUNCTION ===

Функция $\log_{10}(x)$:

Область определения: [0.1; 10.1]

Количество точек: 6

Тип функции: LinkedListTabulatedFunction

Точки функции:

Точка 1: (0,1; -1,000)

Точка 2: (2,1; 0,322)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (8,1; 0,908)

Точка 6: (10,1; 1,004)

Значения функции в различных точках:

$f(-1,0)$ = не определено

$f(0,0)$ = не определено

```
f(0,1) = -1,000 (точное: -1,000)
f(0,3) = -0,868 (точное: -0,523)
f(0,5) = -0,736 (точное: -0,301)
f(1,0) = -0,405 (точное: 0,000)
f(1,5) = -0,074 (точное: 0,176)
f(2,0) = 0,256 (точное: 0,301)
f(3,0) = 0,453 (точное: 0,477)
f(5,0) = 0,690 (точное: 0,699)
f(7,0) = 0,841 (точное: 0,845)
f(10,0) = 1,000 (точное: 1,000)
f(10,1) = 1,004 (точное: 1,004)
f(11,0) = не определено
```

Добавляем точку (3; 0.477):

Количество точек после добавления: 7

Добавляем точку (7; 0.845):

Количество точек после добавления: 8

Удаляем точку с индексом 0:

Количество точек после удаления: 7

Итоговая информация о точках:

Точка 1: (2,1; 0,322)

Точка 2: (3,0; 0,477)

Точка 3: (4,1; 0,613)

Точка 4: (6,1; 0,785)

Точка 5: (7,0; 0,845)

Точка 6: (8,1; 0,908)

Точка 7: (10,1; 1,004)

=== ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ ===

=== ТЕСТИРОВАНИЕ ИСКЛЮЧЕНИЙ ===

1. Тестирование недопустимых индексов:

Поймано исключение: Индекс: 10, Количество: 5

2. Тестирование нарушения порядка X:

Поймано исключение: X должен строго возрастать

3. Тестирование дублирования точек:

Поймано исключение: Точка с $x = 2.0$ уже существует по индексу 1

4. Тестирование удаления при минимальном количестве точек:

Поймано исключение: Невозможно удалить точку – требуется минимум 2 точки

5. Тестирование неверных границ области определения:

Поймано исключение: Левая граница должна быть меньше правой границы: $10.0 \geq 0.0$

6. Тестирование недостаточного количества точек:

Поймано исключение: Требуется как минимум 2 точки

7. Тестирование установки недопустимого X:

Поймано исключение: Точка $x = 0.1$ должно быть больше предыдущей точки $x = 0.5$

8. Тестирование получения точки с неверным индексом:

Поймано исключение: Индекс выходит за пределы: -1

9. Тестирование особых случаев вычисления функции:

$f(-5) = \text{NaN}$

$f(15) = \text{NaN}$

=== ДОПОЛНИТЕЛЬНЫЕ ТЕСТЫ ===

1. Тестирование добавления и удаления точки $(0,0)$:
Ошибка: Точка с $x = 0.0$ уже существует по индексу 2
2. Тестирование добавления и удаления точки $(0,0)$ в `LinkedList`:
Точка $(0,0)$ успешно добавлена в `LinkedListTabulatedFunction`
Последняя точка успешно удалена
3. Тестирование работы с машинным эпсилон:
Точка добавлена (разница больше машинного эпсилона)

